20-Minute Presentation Speech

AetherOS + SpillGuard · Varad Patil (24BIT0175) · to Prof. Harshita Patel · follows the 17-slide deck in order

How to use this. Read the words in normal type out loud, in your own voice — you don't have to memorise them word for word. Orange boxes tell you what to **point at**. [pause] means stop for one breath so Ma'am can look at the slide. Speak slowly: about 130 words a minute. If Ma'am asks a question in the middle, answer it, then say *"Coming back to the slide…"* and continue. The live demo comes **after** slide 17 — use the click-by-click script for that.

#	Slide	Time	Clock	#	Slide	Time	Clock
1	Cover	0:30	0:30	10	Detecting a spill	1:25	11:45
2	The problem	1:30	2:00	11	What a spill looks like	0:55	12:40
3	What I built	0:50	2:50	12	Will this model fit?	1:25	14:05
4	Architecture	1:10	4:00	13	Research method	1:25	15:30
5	CPU scheduling	1:45	5:45	14	Result 1 · cliff	1:25	16:55
6	Memory / paging	1:25	7:10	15	Result 2 · counters	1:25	18:20
7	Deadlock	1:40	8:50	16	Result 3 · remedies	1:10	19:30
8	Journaling	0:55	9:45	17	Conclusion	0:50	20:20
9	The link	0:35	10:20				

1 Cover — AetherOS + SpillGuard

0:00 – 0:30

Good morning, Ma'am. I am Varad Patil, registration number 24BIT0175. My project is **AetherOS plus SpillGuard**.

In one line: AetherOS is an operating-system simulator I built that runs in the browser, so we can actually *see* how a kernel works. SpillGuard is a real tool that watches the GPU memory of this laptop and catches the moment it overflows into normal RAM. Let me start with the problem.

2 The problem — out of VRAM, nothing crashes

0:30 – 2:00

Point at the blue box on the left

Ma'am, every graphics card has its own memory, called **VRAM**. On this laptop it is 6 GB. It is very fast — I measured about **163 gigabytes per second**.

Now, what happens when programs need more than 6 GB? We would expect an error, like "out of memory". But on Windows, nothing crashes. Windows' video memory manager and the NVIDIA driver quietly move the extra data into normal system RAM —

Point at the orange box on the right

— and the GPU has to reach it over the PCIe bus. I measured that at **13.2 gigabytes per second**. That is **12.4 times slower**. [pause]

So the program keeps running, just much slower, and the user has no idea why. This is called a **spill**. Even the programmer cannot see it: I asked CUDA how much memory was free, and it said zero MB — while it kept giving out more memory.

So my questions were: can we detect this from outside, measure how bad it really is, and fix it?

3 What I built — three layers

2:00 – 2:50

Point at each card left to right

I built it in three layers. First, **AetherOS** — this one is simulated, for teaching: two CPU cores with six scheduling algorithms, paging with LRU, FIFO and CLOCK, deadlock detection with Banker's algorithm, and a journaled file system.

Second, **SpillGuard** — this is real. For every app it shows how much is in VRAM and how much has spilled, it detects spills, and it checks whether a new program will fit.

Third, **SpillGuard Repro** — real experiments with an actual AI model, to prove how much a spill hurts and what fixes it. I want to be clear, Ma'am: the kernel is a simulation; everything about the GPU is measured on this laptop.

4 Architecture

2:50 – 4:00

Left box, then the arrow, then right box

This is how it fits together. On the left is the browser. The desktop and apps are built in React and TypeScript, and underneath is my simulated microkernel — the scheduler, memory manager, deadlock engine and file system. It runs on a clock: every 800 milliseconds is one tick, one unit of CPU time.

On the right is the SpillGuard agent, written in Python. Once every second it reads Windows' own performance counters — the same data Task Manager uses — through PDH, DXGI and NVIDIA's NVML library. It analyses them and sends one small JSON message per second to the browser over a WebSocket.

For security, it only listens on this computer, 127.0.0.1, it needs a secret token, and it is read-only. And if the agent isn't running, I can replay a saved recording, so the demo works on any computer.

5 CPU scheduling — Round Robin

4:00 – 5:45

Point at the Core 0 row

Now the OS part. This chart is the real output from my Task Manager. I created three processes, A, B and C, each needing 8 ticks, and selected Round Robin with a time quantum of 4.

Look at Core 0: A runs exactly 4 ticks, then it is preempted and C gets the CPU, then A comes back. That is Round Robin — everyone gets a fair turn. **[pause]**

Point at the B row, then the dashed C boxes

B was put on Core 1 because that queue was shorter — that is load balancing. And these dashed boxes: when B finished, Core 1 had nothing to do, so it **stole** C from Core 0's queue. That is work stealing, which real multi-core schedulers use.

Point at the MLFQ row

The default algorithm is **MLFQ**, multi-level feedback queue: three queues with time slices 4, 8 and 16. A new job starts at the top; if it uses its whole slice, it is CPU-heavy, so it moves down. Interactive jobs stay on top. And every 40 ticks everything is boosted back up, so no process starves — that is aging. I also implemented FCFS, SJF, SRTF and Priority.

6 Memory — paging and page replacement

5:45 – 7:10

Point at the coloured grid

Memory. The simulated RAM has 32 frames of 4 KB. Each process I create needs 16 KB, which is 4 pages. So 8 processes fill the memory completely, and the ninth causes **page faults** — there is no free frame, so the OS must evict someone. These dashed frames are the victims.

Point at the table

Which page to evict depends on the policy. FIFO throws out the page that came in first. LRU throws out the one used least recently. CLOCK, which real operating systems use, gives every page a second chance using a reference bit. And Optimal throws out the page needed farthest in the future — the best possible, but no real system knows the future, so in my app it falls back to LRU, and I label that honestly. On the textbook reference string with three frames, FIFO makes 7 faults and the others make 6.

Point at the orange box

I also show memory protection: if a user program in ring 3 tries to read kernel address 0x0020, it is trapped as a General Protection Fault and the process is stopped.

7 Deadlock — detection and Banker's

7:10 – 8:50

Trace the cycle with your finger, clockwise

Deadlock. A deadlock needs four conditions together: mutual exclusion, hold and wait, no preemption, and circular wait. In my app, "Inject Deadlock" creates exactly this picture: Thread Alpha holds the GPU mutex and waits for the disk mutex; Thread Beta holds the disk mutex and waits for the GPU mutex. Each waits for the other — forever. **[pause]**

To detect it, the engine builds a **wait-for graph** and runs a depth-first search. If the search comes back to a node that is still on its current path, that is a cycle — a deadlock. To recover, we preempt one thread; its lock is released and the other continues.

Point at the P1 … P2 list

On the right is **avoidance**: Banker's algorithm. Each process declares its maximum need; need is maximum minus what it already holds. With 3, 3, 2 available, P1 can finish and return what it holds, so the work vector becomes 5, 3, 2. Then P3, P4, P0 and P2 can finish. Because everyone can finish in some order, the state is **safe** — and a safe state can never deadlock.

8 File system — journaling

8:50 – 9:45

Point at the three boxes, then the power-cut box

The file system. A power cut in the middle of a write can leave files half-written. So I use **write-ahead logging**: every change is first written to a journal as pending, then committed, then checkpointed into the inode table.

When I simulate a power cut, the disk is marked dirty and unmounted, with one torn write still pending. Then **fsck** reads the journal: anything that never committed is rolled back, anything committed but not applied is replayed. The disk is clean again and never half-written — the same idea as ext4 and NTFS journaling.

9 The link — same problem, one level up

9:45 – 10:20

Read across the table rows

And this is why the two halves belong together, Ma'am. Everything AetherOS teaches exists inside a real GPU. VRAM is like physical frames. Shared GPU memory in system RAM is like swap space. Windows' video memory manager evicting allocations is page replacement. And a spill is basically **thrashing on the GPU**. So SpillGuard is the real-world version of the same memory-management problem.

10 SpillGuard — detecting a spill

10:20 – 11:45

Point at OK → PRESSURED → SPILLING

So how do I detect a spill? Once a second, for every process, the agent checks three things. First, is the GPU at least 92 percent full? That is "pressured". Second, has this process's shared memory grown more than 128 MB above its own normal baseline? And third, is it still rising? If all three hold for two seconds in a row, it is "spilling". It goes back to OK only after three clear seconds, so it doesn't flicker.

The baseline matters because browsers always keep some shared memory; without it, they would be flagged all the time. It also names the **culprit** — the process whose memory grew most in the last 30 seconds.

Point at the four numbers

I tested it against independent ground truth, by measuring the speed of every memory block. It caught **all 3** real spills, gave **zero** false alarms on runs that came close at 98 percent, detected in about **one second**, and uses only **2.3 percent** of one CPU core.

11 What a spill looks like

11:45 – 12:40

Point at the red banner, then the python.exe bar

This is what it looks like, redrawn from a recording on this laptop. A test program kept allocating GPU memory. VRAM reached 5.81 of 5.86 GB — completely full — and then 960 MB of that program's data ended up in system RAM; that is the orange part of the bar.

The banner turns red and names the process, and the culprit line says python.exe grew its memory by 3.3 GB just before. In the live demo I will replay this exact recording.

12 Will this model fit? Banker's on real VRAM

12:40 – 14:05

Point at M = W + KV + B + O

Detection is good, but prevention is better. Before launching an AI model, SpillGuard estimates its GPU memory from the model file: the weights, the KV cache — which stores past tokens and grows with the context length — the buffers, and the CUDA context. For this model it is 2.64 GB. I calibrated it on this laptop, and the error is only **46 MB**, while Ollama's own estimate was off by about **900 MB**.

Point at SAFE, then DENY

Then I use Banker's algorithm with VRAM as the resource. Here is the interesting part: for a new program claiming 5.5 GB, the textbook Banker's says **SAFE**, because it assumes every process eventually finishes and gives its memory back. But a browser or a game never finishes. So my version reserves their remaining need and says **DENY** — launching it would cause a spill. So the textbook algorithm is too optimistic for long-running programs.

13 Research method — a controlled experiment

14:05 – 15:30

Point at Baseline → Pressure → Recovery

Now the research. I wanted to prove how much a spill actually hurts a real application. I used a real local AI model, Qwen 2.5 with 3 billion parameters, running in Ollama. Every trial has three stages: **baseline** — the model alone; **pressure** — the same work while a second program, which I call the co-tenant, holds GPU memory; and **recovery** — alone again. Same prompt, 128 tokens, temperature zero, so the output must be exactly the same every time. I measure milliseconds per generated token.

Point at the three verdict cards

A trial counts as **FAIL** only if it is at least twice as slow *and* fully recovers afterwards — that proves the pressure caused it. **PASS** is a clean trial that stayed under 2×. And anything doubtful — for example the laptop ran short of RAM, or the output changed — is **INCONCLUSIVE**, and it stays in the results, never hidden. Every run is saved with SHA-256 hashes, so the data cannot be changed later without it showing.

14 Result 1 — a cliff at the VRAM boundary

15:30 – 16:55

Move your finger left to right along the chart

Result one. On the x-axis is how far total memory demand goes past the 6 GB. At minus 512 — everything fits — the model is only 1.16 times slower, just from sharing the GPU. Exactly at capacity, 1.21. **[pause]**

But just 256 MB over, it jumps to **2.36 times slower** — from 14 milliseconds per token to 33. And 512 over is no worse. So it is not a gradual slope; it is a **cliff** right at the VRAM boundary. Every failure recovered fully once the co-tenant left, with the same output.

Point at the small square

And this little square is an idle co-tenant holding the same memory — almost no slowdown. That is the next slide.

15 Result 2 — same counters, opposite outcomes

16:55 – 18:20

Point at the two columns of the table

This is my most interesting finding, Ma'am. At plus 256 MB, I compared an **active** co-tenant, which touches its memory every quarter of a second, with an **idle** one holding exactly the same memory. Look at the Windows counters: identical. Same memory for the model, same for the co-tenant, GPU full in both cases.

Point at 2.37× and 1.03×

But the active one makes the model **2.37 times slower**, and the idle one only **1.03**. **[pause]** So memory counters alone cannot tell harmful pressure from harmless pressure. Even my own detector, which uses counters, caught all 4 real slowdowns but raised 3 alarms on harmless runs. What does separate them is an **intervention** — changing one thing and measuring again.

My guess is that an active co-tenant makes Windows keep moving pages back and forth, which is called churn — but that is a hypothesis; proving it needs ETW tracing.

16 Result 3 — which fix works?

18:20 – 19:30

Go down the bars top to bottom

Result three: what fixes it? First I re-ran the failing case from scratch — it failed again, so overall **6 out of 6** trials failed across two separate runs. Then I tested each fix as its own experiment.

Releasing all of the co-tenant's memory: fixed. Running it after the AI model: fixed. Releasing half, or even just one eighth: fixed, because that brings us back under capacity.

Point at the highlighted last row

But the best one is the last: the co-tenant keeps every byte and just **pauses its activity** — and the model is back to full speed, 1.01. That is the least disruptive fix.

17 Conclusion

19:30 – 20:20

Point at Built, Found, Limits · next

To conclude, Ma'am. I built AetherOS, which makes scheduling, paging, deadlock and journaling visible and testable, and SpillGuard, which detects real GPU memory spills and does admission control with Banker's algorithm.

I found that crossing VRAM by just 256 MB makes a real AI model 2.36 times slower, that memory counters cannot tell harmful pressure apart, and that pausing the other program fixes it.

The limits, honestly: one laptop, two trials per level, and the churn mechanism is not proven yet. Next, I want to verify it with ETW tracing and test more GPUs and models. Everything is tested — 66 front-end tests and 149 agent tests pass.

Thank you, Ma'am. **If you'd like, I can show you the live demo now.**

After the speech: switch to the browser tab opened by `START_DEMO.bat` and follow **AetherOS_Demo_Script.pdf** (Task Manager → Memory Map → Deadlock → Terminal → Finder → SpillGuard → spill replay → evidence desk). For questions, keep **AetherOS_SpillGuard_Complete_Guide.pdf** open at section 17 (viva Q&A) and section 19 (numbers cheat-sheet).